



Dizajn sistema

Alat za digitalni menadžment radnika i resursa

Aleksandra Vučićević, Jelena Maletić, Anastasija Mitrić,
Željka Nikolić, Konstantin Šarić, Jovana Šikanjić

Sadržaj

1 Uvod.....	3
1.1 Namjena sistema.....	3
1.2 Projektni ciljevi.....	3
1.3 Definicije i skraćenice.....	4
1.4 Referentni dokumenti.....	5
1.5 Kratak pregled dokumenta i predložene arhitekture.....	5
2 Arhitektura postojećeg sistema.....	5
2.1 Kratak pregled arhitekture i funkcionalnosti podsistema.....	6
2.1.1 Bekend podsistem i baza podataka.....	7
2.1.2 Klijentski podsistemi: veb i desktop.....	8
2.2 Dekompozicija sistema.....	9
2.3 HW/SW mapiranje.....	11
2.3.1 Dijagram komponenata.....	11
2.3.2 Dijagram razmještaja.....	12
2.4 Perzistentni sloj.....	13
2.5 Kontrola prava pristupa.....	13
2.6 Identifikacija konkurentnosti.....	15
2.7 Kontrola toka.....	16
2.8 Granična stanja sistema.....	16
2.8.1 Inicijalizacija.....	16
2.8.2 Terminiranje.....	17
2.8.3 Otkazi.....	17

1 Uvod

Svrha ovog dokumenta je da detaljno opiše dizajn **HEFEST** - alat za digitalni menadžment radnika i resursa sistema i da dopuni specifikaciju softverskih zahtjeva.

1.1 Namjena sistema

HEFEST je veb aplikacija namijenjena digitalizaciji procesa upravljanja ljudstvom i materijalnim resursima preduzeća. Sistem omogućava centralizovanu kontrolu nad planiranjem i realizacijom radnih aktivnosti.

Ključne funkcionalnosti sistema su:

- **Upravljanje zadacima:** Planiranje, dodjela i praćenje statusa dnevnih zadataka i projekata.
- **Kontrola resursa:** Evidencija i raspoređivanje opreme, vozila i materijala po projektima.
- **Administracija korisnika:** Upravljanje nalogima i definisanje nivoa pristupa za različite profile zaposlenih (direktor, knjigovođa, magacioner, poslovođa, tehničar).
- **Izveštavanje:** Automatizovana izrada dnevnih i sumarnih izvještaja o radu i potrošnji resursa.

Sistem je projektovan da poveća operativnu efikasnost i obezbijedi precizan uvid u stanje na terenu u realnom vremenu.

1.2 Projektni ciljevi

- **Sigurnost**

Zaštita korisničkih podataka ostvarena je kroz autentifikaciju, autorizaciju, kriptografsku zaštitu lozinki, te kontrolu pristupa ostvarenu kroz primjenu principa najmanjih privilegija, gdje svaki korisnik ima pristup isključivo onim funkcionalnostima i podacima neophodnim za obavljanje njegovih radnih zadataka. Validacija ulaznih podataka, kojom se osigurava konzistentnost baze podataka, predstavlja još jedan sloj zaštite koji osigurava stabilnost sistema i sprečava maliciozne pokušaje manipulacije podacima.

- **Laka upotreba**

Sistem je osmišljen s ciljem da bude intuitivan i jednostavan za korištenje, bez potrebe za dodatnom obukom korisnika. Moderan grafički interfejs, pregledna navigacija, logično organizovan sadržaj i jasne kontrole doprinose prijatnom korisničkom iskustvu, bez obzira na tehnička znanja korisnika.

- **Prenosivost**

Korištenjem savremenih web tehnologija osigurava se kompatibilnost sa najnovijim verzijama popularnih web čitača na različitim desktop i laptop uređajima. Arhitektura sistema

je optimizovana za rad u standardnim rezolucijama, čime se obezbeđuje stabilnost prikaza i dostupnost ključnih funkcionalnosti na radnim stanicama korisnika.

- **Prilagodljivost**

Sistem omogućava visok stepen prilagođavanja individualnim potrebama korisnika kroz podršku za tamni i svijetli režim rada (light/dark mode), kao i različite modalitete prikaza podataka (list/grid). Fleksibilnost je dodatno osigurana kroz alternativne načine interakcije korisnika sa sadržajem, te napredno filtriranje sadržaja, čime se radni proces optimizuje prema preferencijama korisnika.

- **Proširivost sistema**

Arhitektura sistema je modularna i fleksibilna, što omogućava jednostavno dodavanje novih funkcionalnosti. Jasno razdvajanje poslovne logike, korisničkog interfejsa i baze podataka olakšava održavanje i dalji razvoj. Dobro definisani API endpoint-i omogućavaju brzu integraciju dodatnih komponenti bez potrebe za značajnim promjenama u postojećem kodu.

- **Lako održavanje**

Radi lakšeg održavanja, primijenjene su dobre prakse u programiranju, što podrazumijeva kod strukturiran u skladu sa odabranim arhitekturnim stilom, te izradu projektne dokumentacije. Redovno ažuriranje sistema i pravovremeno otklanjanje problema ključni su za osiguranje stabilnosti i pouzdanosti aplikacije.

- **Dobra dokumentacija**

Projektni ciklus prati sveobuhvatna tehnička dokumentacija koja uključuje specifikaciju zahtjeva (SRS), detaljan dizajn sistema i precizan objektni model. Ovakav pristup osigurava transparentnost arhitekturnih odluka, olakšava proces održavanja i omogućava brzu obuku novih članova tima kroz jasno definisane strukture i funkcionalnosti sistema.

1.3 Definicije i skraćenice

Skraćenica	Objašnjenje
HTTPS	Hypertext Transfer Protocol Secure
API	Application Programming Interface
JPA	Java Persistence API
SQL	Structured Query Language
ORM	Object - Relational Mapping
CRUD	Create, Read, Update, Delete

1.4 Referentni dokumenti

Referentni dokumenti su:

- specifikacija softverskih zahtjeva **HEFEST** - alat za digitalni menadžment radnika i resursa sistema
- <https://www.geeksforgeeks.org/spring-boot-architecture/>
- https://www.researchgate.net/figure/Fig-2-Architecture-flow-of-spring-boot-Application-s-Spring-boot-uses-all-the-features_fig2_341151097
- <https://docs.spring.io/spring-boot/docs/3.2.5/reference/htmlsingle/>

1.5 Kratak pregled dokumenta i predložene arhitekture

Ovaj dokument pruža pregled arhitekture **HEFEST** sistema, uključujući dekompoziciju kroz dijagram paketa. Takođe su prikazani dijagrami komponenata i dijagrami razmještaja, koji oslikavaju međusobne veze između modula i infrastrukture sistema. Perzistentni sloj predstavljen je konceptualnim modelom na visokom nivou apstrakcije.

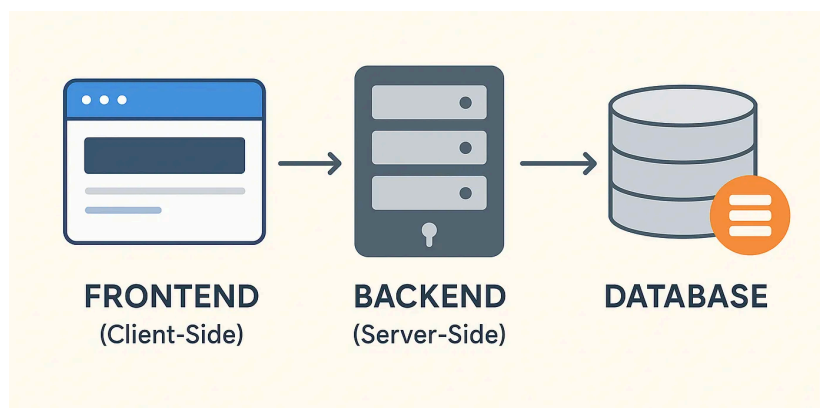
Prava pristupa funkcionalnostima sistema variraju u zavisnosti od tipa korisnika (administrator, knjigovođa, magacioner, tehničar, poslovođa), a ta ograničenja su prikazana u tabeli prava pristupa. Kako sistem omogućava istovremeni rad više korisnika, ta tematika je detaljno obrađena u posebnoj sekciji. Kontrola toka sistema – od prijave korisnika do izvršenja zadataka, takođe je opisana u posebnom dijelu dokumenta. Na kraju, dokument razmatra i granične slučajeve upotrebe sistema.

2 Arhitektura postojećeg sistema

U procesu projektovanja sistema, arhitektura predstavlja temeljni okvir koji definiše strukturu komponenti i njihovu međusobnu komunikaciju, osiguravajući punu usklađenost sa projektnim ciljevima. Posebna pažnja posvećena je definisanju interfejsa za interakciju sa krajnjim korisnicima, uz strogo poštovanje standarda pouzdanosti, sigurnosti prenosa podataka i operativne stabilnosti.

U narednim sekcijama detaljno su analizirani ključni aspekti arhitekture, uključujući izbor tehnologija, projektnih obrazaca, sigurnosne mjere i mogućnosti skaliranja. Ova analiza omogućava razumijevanje donesenih odluka i njihovog direktnog uticaja na funkcionalnost, performanse i kvalitet konačnog softverskog rješenja.

2.1 Kratak pregled arhitekture i funkcionalnosti podsistema

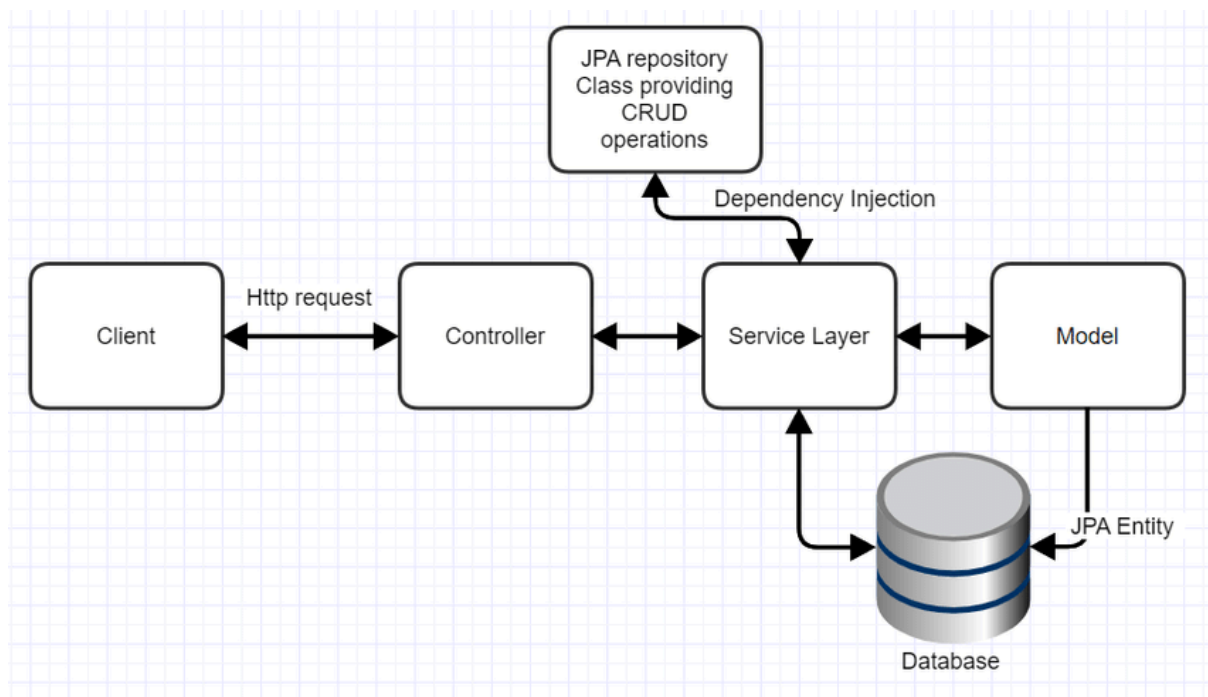


<< Arhitektura sistema >>

Sistem je zasnovan na proširenoj **troslojnoj arhitekturi** (*3-tier architecture*), koja omogućava jasnu separaciju odgovornosti, lakše održavanje, bolju skalabilnost sistema, te hibridni klijentski pristup. Ovakav dizajn dijeli sistem na tri logički odvojena segmenta:

- **Sloj prezentacije (*Presentation Layer*)**: Predstavlja korisnički interfejs i komunikacioni sloj putem kojeg korisnik vrši interakciju sa sistemom. Njegova primarna uloga je vizuelni prikaz informacija i prikupljanje korisničkih unosa za dalju obradu. Obuhvata dva tehnički različita klijenta: **veb aplikaciju (React)** za operativne korisnike i **desktop aplikaciju (Java Swing)** za administraciju sistema.
- **Sloj aplikativne logike (*Business Logic Layer*)**: Predstavlja vezu između korisničkog interfejsa i baze podataka. Unutar ovog sloja definisana su sva poslovna pravila i logika sistema, čime se osigurava da podaci budu obrađeni u skladu sa zahtjevima klijenata prije nego što se sačuvaju ili prikažu. Obrađuje zahtjeve oba tipa klijenata.
- **Sloj podataka (*Data Layer*)**: Odgovoran je za sigurno skladištenje i upravljanje podacima. Korišćenjem relacione baze podataka, ovaj sloj garantuje integritet i trajnost zapisa, čineći ih dostupnim za sve operacije unutar sistema.

2.1.1 Bekend podsistem i baza podataka



<< Arhitektura Spring Boot aplikacija >>

Spring Boot je dodatak **Spring framework**-u koji olakšava i automatizuje pravljenje Spring baziranih aplikacija. Glavna prednost je **Auto-configuration** i **Embedded Tomcat** server, što omogućava da se aplikacija pokrene odmah bez eksternih podešavanja.

Bekend podsistem, implementiran kao Spring Boot aplikacija, prati slojevit, 4-slojnu (4-Tier) arhitekturu, koja osigurava jasno odvajanje odgovornosti među slojevima, lakše testiranje i održavanje koda.

Odgovornosti slojeva prikazanih na dijagramu:

- **Client (Klijent)**: Predstavlja korisnički interfejs koji šalje HTTP zahtjeve ka aplikaciji upotrebom REST API standarda.
- **Controller (Kontroler)**: Služi kao ulazna tačka za **web klijenta**, prihvata zahtjeve koje on šalje, vrši osnovnu validaciju ulaznih podataka, te delegira izvršavanje poslovne logike servisnom sloju.
- **Service Layer (Servisni sloj)**: Središnji sloj aplikacije u kojem je smještena sva poslovna logika. Koordinira rad između repozitorijuma i modela, primjenjujući definisana pravila poslovanja. Dok kontroleri delegiraju zahtjeve web klijenta ovom sloju, **administratorski modul**, zahvaljujući integraciji sa **Spring** kontekstom, direktno konzumira ove servise.
- **JPA Repository (JPA repozitorijum)**: Koristi Spring Data JPA za apstrakciju pristupa bazi podataka. Putem mehanizma **Dependency Injection**, repozitorijum se ubrizgava u servisni sloj, omogućavajući CRUD operacije bez potrebe za pisanjem SQL upita.
- **JPA Entity/Model**: Predstavlja domenske objekte aplikacije, koje su mapirane na tebele u bazi podataka pomoću ORM alata (Hibernate/JPA), definišući koje strukturu podataka koji se pohranjuju.

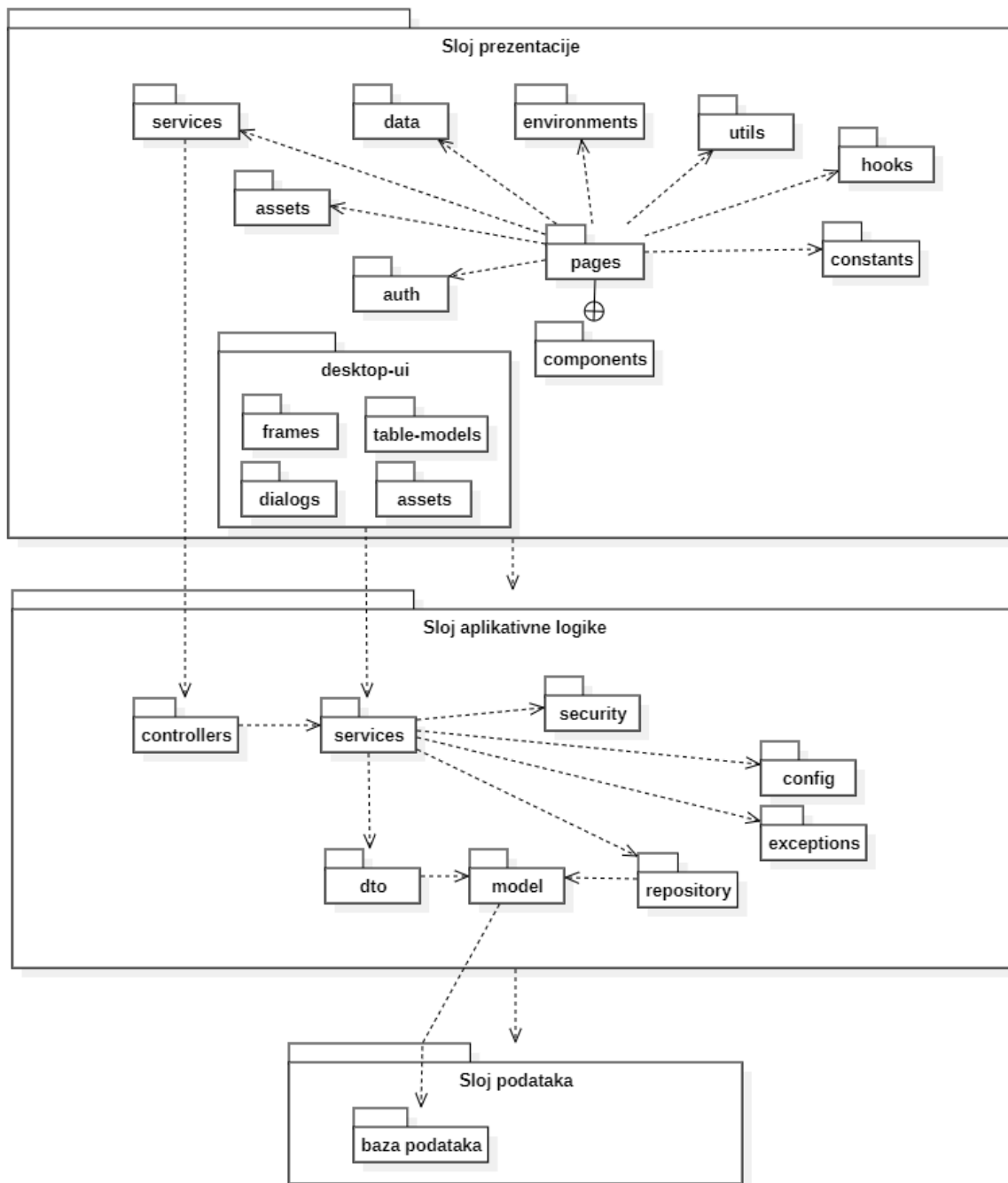
- **Database (Baza podataka):** Perzistentni sloj (MySQL relacionalna baza podataka) u kojem se podaci trajno skladište.

2.1.2 Klijentski podsistemi: veb i desktop

HEFEST sistem nudi dva specijalizovana interfejsa prilagođena različitim potrebama korisnika:

- **Veb klijent:** Implementiran kao moderna **Single Page React** aplikacija uz podršku **Vite** alata. Ovaj klijentski podsistem prati modernu komponentno-baziranu arhitekturu, što omogućava da se kompleksan interfejs razbije na manje, nezavisne i višekratno upotrebljive cjeline, što značajno olakšava održavanje koda i ubrzava razvoj. React-ov deklarativni stil i efikasno upravljanje stanjima osiguravaju da aplikacija bude brza, predvidljiva i laka za testiranje, dok Vite, kao alat za razvoj i izgradnju, koristi prednosti izvornih ES modula, omogućavajući gotovo trenutno pokretanje razvojnog servera i *hot reload* (osvježavanje izmjena u realnom vremenu). Na taj način su optimizovane performanse same aplikacije u produkciji.
- **Desktop klijent:** Ovaj klijentski podsistem, zasnovan na **Java Swing** biblioteci, razvijen je za potrebe naprednog upravljanja **HEFEST** sistemom. Omogućava izolaciju kritičnih administratorskih funkcija, koje podrazumijevaju brz pregled i manipulaciju svim korisničkim nalogima, izvan standardnog veb pregledača.

2.2 Dekompozicija sistema



Arhitektura sistema je predstavljena pomoću dijagrama paketa i bazira se na troslojnom arhitekturnom modelu. Sistem je podijeljen u tri osnovna hijerarhijska sloja: prezentacioni sloj, sloj aplikativne logike i sloj podataka. Osim ove vertikalne podjele, izvršena je i horizontalna dekompozicija sistema, kojom su dodatno razdvojeni segmenti unutar svakog sloja.

Sloj prezentacije

Sloj prezentacije je zadužen za interakciju sa korisnicima sistema (administrator, direktor, poslovođa, tehničar, magacioner) i vizuelnu reprezentaciju resursa firme. Specifičnost ovog sloja je podjela na dva nezavisna klijentska rješenja koja dijele zajedničku infrastrukturu aplikativne logike:

- **Administratorski desktop modul (desktop-ui):** Poseban paket realizovan u **Java Swing** tehnologiji. Sadrži podpakete **frames** (glavni prozori), **dialogs** (modalni prozori za unos podataka) i **table-models** (specijalizovani sloj za povezivanje podataka sa vizuelnim tabelama). Ovaj modul posjeduje i sopstveni **assets** paket za lokalne grafičke resurse.
- **Veb klijent:** Namijenjen operativnom radu zaposlenih firme. Sastoji se od sljedećih podslojeva:
 - **Sloj ekrana i komponenti (pages i components):** Predstavlja najviši nivo prezentacionog sloja. Podsloj pages definiše cjelovite prikaze, dok components sadrži višekratne UI elemente. Odgovornosti ovog sloja su renderovanje podataka i upravljanje korisničkim akcijama.
 - **Sloj klijentske logike i stanja (hooks i data):** Koristi prilagođene kuke (**Custom Hooks**) za upravljanje stanjem aplikacije i asinhronim operacijama. Podsloj data definiše strukture koje se koriste unutar klijentske aplikacije.
 - **Sloj komunikacije sa slojem aplikativne logike (services):** Ovaj podsloj je zadužen za slanje HTTP zahtjeva ka REST API-ju. On izoluje mrežnu logiku od UI komponenti, vršeći transformaciju primljenih podataka u format pogodan za prikaz.
 - **Sloj podrške i okruženja (assets, utils i environments):** Obuhvata statičke resurse (ikonice, slike), pomoćne funkcije za formatiranje podataka, kao i konfiguracione parametre specifične za okruženje u kojem aplikacija radi (npr. URL backend servera).

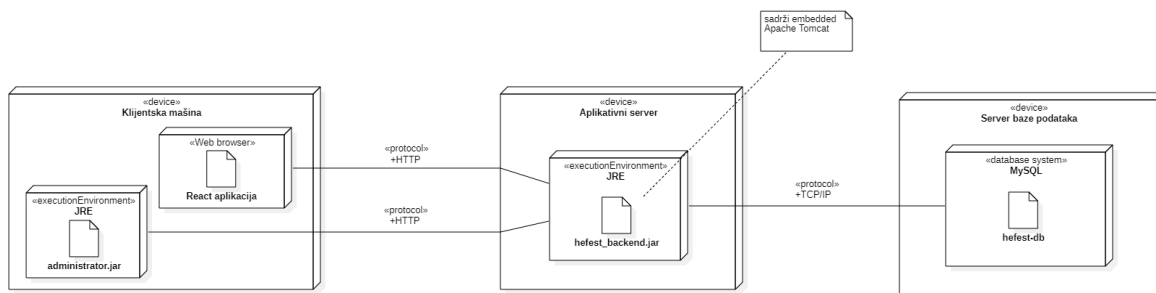
Sloj aplikativne logike

Sloj aplikativne logike, razvijen u Spring Boot razvojnom okviru, predstavlja jezgro sistema gdje se implementira poslovna logika.

- **Kontrolerski sloj (controller):** Ulazna tačka za sve zahtjeve iz sloja prezentacije. Vršiti mapiranje HTTP ruta na odgovarajuće metode, upravlja ulaznom validacijom i koristi **exceptions** paket za globalno presretanje grešaka.
- **Sloj servisa (services):** Služi kao univerzalni procesor, odnosno predstavlja centralnu tačku sistema koja uslužuje oba klijenta. Dok veb klijent pristupa servisima indirektno putem **kontrolera**, administratorski desktop modul ostvaruje direktnu zavisnost ka servisnom sloju. Ovakva dekompozicija omogućava da se poslovna logika piše i održava na jednom mjestu, dok se samo način prezentacije prilagođava tipu aplikacije. Servisne klase koordiniraju rad između repozitorijuma i koriste DTO (**Data Transfer Objects**) za prenos podataka ka višim slojevima, čime se postiže enkapsulacija domenskih modela.
- **Sloj sigurnosti (security):** Implementira mehanizme autentifikacije i autorizacije. Zadužen je za zaštitu resursa sistema na osnovu uloga korisnika.

- **Veb aplikacija:** Namijenjena je operativnim korisnicima i sastoji se od dvije podkomponente:
 - **GUI:** služi za renderovanje interfejsa i direktnu interakciju s korisnikom.
 - **Client logic:** služi za obradu podataka prije nego što se prikažu na grafičkom interfejsu, upravljanje stanjem i pristup REST API endpoint-ima.
- **Administratorska aplikacija:** Realizovana je kao desktop klijent (**Java Swing**) i zadužena je za inkapsuliranje logike za upravljanje korisničkim nalogima, koristeći specijalizovane modele za prikaz podataka u tabelarnom formatu.
- **Aplikativni server:** Predstavlja centralnu komponentu koja putem REST API interfejsa izlaže poslovnu logiku. Unutar njega, podkomponenta **Kontroleri** upravlja ulaznim zahtjevima, dok podkomponenta **Servisi** implementira konkretna poslovna pravila. **Perzistentni sloj** obezbjeđuje komunikaciju sa **Serverom baze podataka**, garantujući integritet podataka kroz MySQL sistem.

2.3.2 Dijagram razmještaja



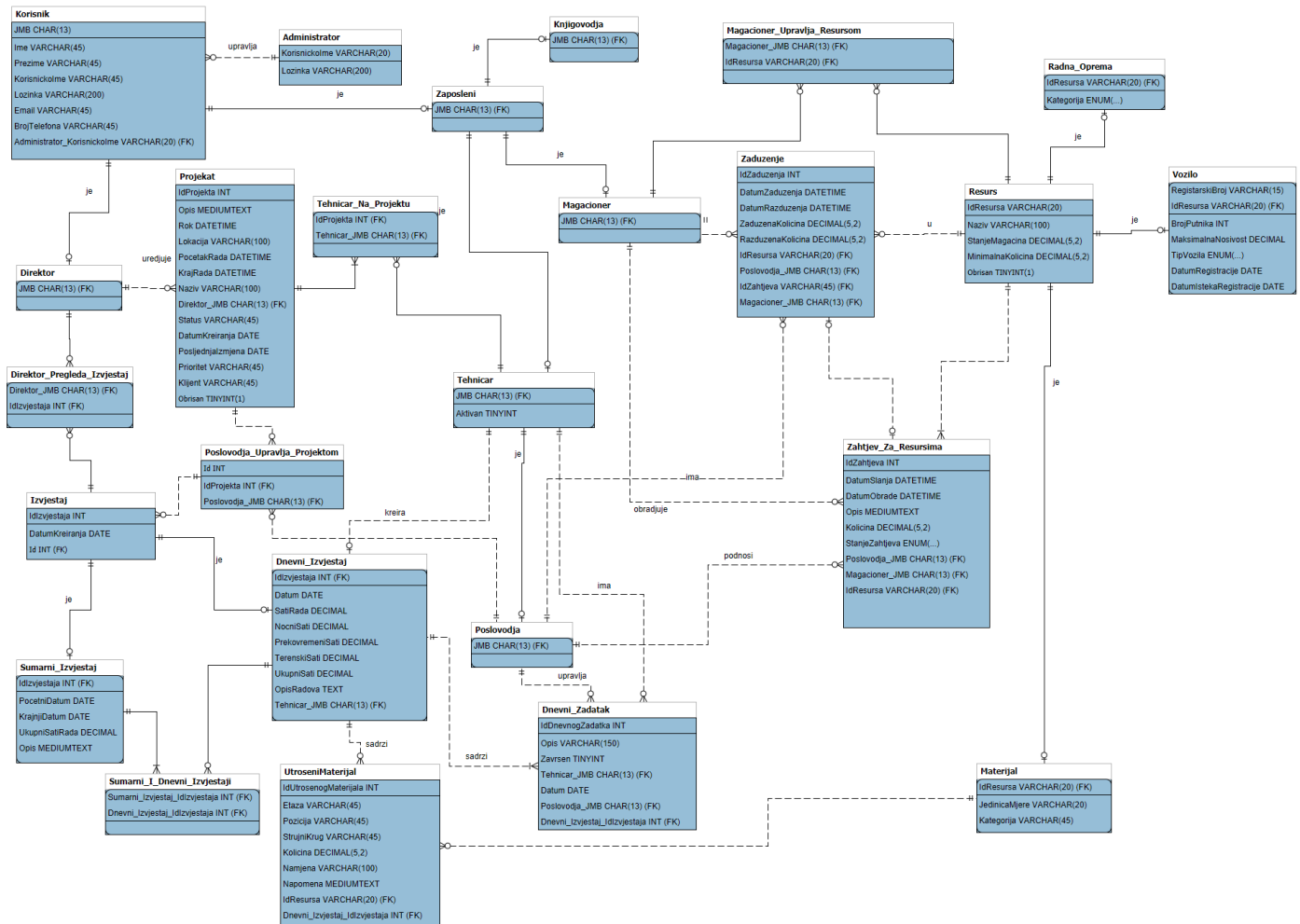
<<Dijagram razmještaja>>

Dijagram razmještaja prikazuje fizičku konfiguraciju sistema **HEFEST**, definišući topologiju hardverskih čvorova i softverskih artefakata koji se na njima izvršavaju. Sistem je dekomponovan na tri primarna čvora, uz specifičnu strukturu klijentske strane:

- **Klijentska mašina:** Predstavlja korisničko okruženje koje podržava dva različita načina interakcije sa sistemom, u zavisnosti od uloge koju klijent ima u sistemu:
 - **Veb pregledač:** Unutar veb pregledača se izvršava *React aplikacija*, namijenjena operativnom radu zaposlenih, te je distribuirana kao artefakt koji komunicira sa serverskom stranom putem **HTTP** protokola.
 - **Desktop okruženje:** Namjenski sloj za izvršavanje *administrator.jar* artefakta. Ova **Java Swing** aplikacija omogućava administratoru sistema direktno upravljanje sistemom van veb okruženja, te sa serverskom stranom komunicira putem **HTTP** protokola.
- **Aplikativni server:** Centralni čvor logike sistema. Zasnovan je na **JRE** okruženju u kojem se izvršava *hefest_backend.jar*. Specifičnost ovog čvora je upotreba **Embedded Apache Tomcat** servera, koji je integrisan unutar samog izvršnog JAR fajla, čime se pojednostavljuje proces razmještaja i pokretanja pozadinskih servisa.
- **Server baze podataka:** Namjenski čvor za perzistenciju podataka koji koristi **MySQL** sistem za upravljanje bazama podataka. Komunikacija između aplikativnog servera i baze podataka ostvaruje se putem **TCP/IP** protokola (koristeći JDBC drajver), čime se obezbjeđuje integritet i siguran prenos informacija.

Ovakva dekompozicija omogućava skalabilnost sistema, jer se svaki od navedenih čvorova može nezavisno konfigurirati i izmjestiti na posebne fizičke servere u produkcijskom okruženju.

2.4 Perzistentni sloj



<<Konceptualni model>>

Prethodna slika predstavlja konceptualni model baze podataka. Na osnovu njega implementiran je perzistentni sloj unutar aplikacije, koji se koristi radi očuvanja podataka i nakon ponovnog pokretanja aplikacije. Kao sistem za upravljanje relacionim bazama podataka odabrali smo MySQL.

2.5 Kontrola prava pristupa

Pristup funkcionalnostima sistema **HEFEST** zasnovan je na ulogama korisnika (*Role-Based Access Control* - *RBAC*). Svaka uloga ima definisan skup privilegija nad entitetima sistema, čime se osigurava integritet podataka i sigurnost poslovanja.

- **Administrator** je zadužen za upravljanje korisničkim nalogima.
- **Direktor** posjeduje najširi spektar ovlaštenja nad projektima i uvid u rad zaposlenih.
- **Operativne uloge** (Tehničar, Magacioner, Poslovođa) imaju pristup isključivo onim metodama koje su neophodne za uspješno izvršavanje obaveza koje imaju u skladu sa svojom ulogom.

Učesnici Klase	Administrator	Direktor	Magacioner	Knjigovođa	Tehničar	Poslovođa
KorisnickiNalog	kreiranjeKorisnickogNaloga() azuriranjeKorisnickogNaloga() brisanjeKorisnickogNaloga() pregledKorisnickihNaloga() pregledKorisnickogNaloga() pretragaKorisnickihNaloga()	prijava() odjava() promjenaLozinke()	prijava() odjava() promjenaLozinke()	prijava() odjava() promjenaLozinke()	prijava() odjava() promjenaLozinke()	prijava() odjava() promjenaLozinke()
Zaposleni	/	pregledZaposlenih() pretragaZaposlenih() pregledZaposlenog() brisanjeZaposlenog()	/	pregledZaposlenih() pretragaZaposlenih() pregledZaposlenog()	/	/
Tehnicar	/	/	/	/	/	pregledListeTehnicara() pretragaTehnicara() pregledPodatakaOTehnicaru()
DnevniStatus	/	/	/	/	azuriranjeDnevnogStatusa()	/
Projekat	/	pregledProjekata() pretragaProjekata() pregledProjekta() dodavanjeProjekta() azuriranjeProjekta() brisanjeProjekta()	/	/	/	pregledDodijeljenihProjekata() pretragaDodijeljenihProjekata() pregledInformacijaOProjektu()
DnevniZadatak	/	/	/	/	pregledDnevnihZadataka() pretragaDnevnihZadataka() pregledDnevnogZadatka() promjenaStanjaDnevnogZadatka()	/
DnevniIzvjestaj	/	pregledDnevnihIzvjestaja() pretragaDnevnihIzvjestaja() pregledDnevnogIzvjestaja()	/	pregledDnevnihIzvjestaja() pretragaDnevnihIzvjestaja() pregledDnevnogIzvjestaja()	kreiranjeIzvjestaja()	validacijaDnevnogIzvjestaja()
SumarniIzvjestaj	/	pregledSumarnihIzvjestaja() pretragaSumarnihIzvjestaja() pregledSumarnogIzvjestaja()	/	pregledSumarnihIzvjestaja() pretragaSumarnihIzvjestaja() pregledSumarnogIzvjestaja()	/	generisanjeSumarnogIzvjestaja() pregledSumarnihIzvjestaja() pretragaSumarnihIzvjestaja() pregledSumarnogIzvjestaja()

Zahtjev Za Resursima	/	/	pregledListeZahtjeva() pretragaZahtjeva() pregledZahtjeva() PromjenaStanjaZahtjeva()	/	/	kreiranjeZahtjeva() pregledKreiranihZahtjeva() pretragaZahtjeva() pregledZahtjeva()
Resurs	/	/	pregledResursa() pretragaResursa() dodavanjeResursa() brisanjeResursa() azuriranjeResursa() pregledZaduzenihResursa() pretragaZaduzenihResursa() pregledZaduzenja() pretragaZaduzenja() dodavanjeZaduzenja() azuriranjeZaduzenja() brisanjeZaduzenja()	/	/	/
Kalendar	/	pregledKalendara()	/	/	pregledKalendara()	pregledKalendara()
Mapa	/	pregledMapeSaRadilistima() dodavanjeRadilista() azuriranjeRadilista() brisanjeRadilista()	/	/	/	pregledMapeSaRadilistima()

2.6 Identifikacija konkurentnosti

S obzirom da će **HEFEST** aplikaciju koristiti više različitih tipova korisnika (tehničari, poslovođe, magacioneri, knjigovođe, direktori i administratori), identifikacija i obrada konkurentnosti predstavlja ključni aspekt za očuvanje stabilnosti i performansi sistema. Sistem je dizajniran tako da omogući istovremeno korišćenje aplikacije od strane više korisnika, bez konflikta u pristupu podacima ili funkcionalnostima.

Za svaku korisničku ulogu implementirani su jasno odvojeni funkcionalni tokovi i prava pristupa, čime se mogućnost preklapanja zahtjeva svodi na minimum i povećava sigurnost pristupa informacijama. Na primjer, dok tehničari unose izvještaje sa terena, magacioner ažurira stanje zaliha, a poslovođe upravljaju dnevnim i sumarnim izvještajima – sve to može da se odvija paralelno, bez međusobnog ometanja.

Baza podataka se nalazi na serverskoj strani sistema, i koristi se mehanizam za transakcioni pristup podacima kako bi se očuvala konzistentnost čak i u uslovima istovremenih izmjena. Korišćenje Spring Boot arhitekture omogućava slojevitou organizaciju sistema, što znači da se svaki funkcionalni dio može razvijati, testirati i nadograđivati nezavisno, bez ugrožavanja rada ostalih komponenti.

Pored toga, sistem podržava skalabilnost, što omogućava njegovo prilagođavanje većem broju korisnika u budućnosti, kao i integraciju novih uloga i funkcionalnosti po potrebi. Na

ovaj način, **HEFEST** aplikacija obezbeđuje stabilno i bezbjedno radno okruženje za sve korisnike, uz podršku paralelnog izvršavanja ključnih operacija.

2.7 Kontrola toka

U našem Spring Boot sistemu, kontrola toka aplikacije centralizovana je i zasnovana na događajima. Korisnička interakcija se odvija putem grafičkog korisničkog interfejsa, gdje korisnici biraju različite opcije ili popunjavaju forme, čime se generišu događaji. Ti događaji se obrađuju asinhrono unutar aplikacije, što omogućava njen nesmetan rad bez čekanja na završetak svake pojedinačne akcije.

Kada korisnik inicira neku radnju, Spring Boot aplikacija prima odgovarajući događaj i automatski pokreće relevantnu logiku. Asinhroni pristup obradi omogućava veću skalabilnost i odziv sistema, posebno u okruženjima sa velikim brojem korisnika.

Za upravljanje tokovima zahtjeva i odgovora oslanjamo se na mehanizme ugrađene unutar Spring Boot-a, koristeći API View komponente i standardne HTTP metode u okviru RESTful arhitekture. Ovakav pristup pojednostavljuje razvoj, održavanje i širenje servisa, a omogućava i pouzdano upravljanje višestrukim korisničkim zahtjevima – bilo da dolaze od tehničara ili poslovođe.

2.8 Granična stanja sistema

2.8.1 Inicijalizacija

HEFEST sistem se sastoji od više međusobno povezanih podsistema (frontend i backend) od kojih se svaki može nezavisno pokrenuti. Inicijalizacija svakog podsistema obuhvata pripremu internih struktura podataka i potrebnih resursa. Prilikom prvog pokretanja potrebno je izvršiti inicijalnu konfiguraciju na serverskim mašinama, ukoliko već nije postavljena.

Za potpunu funkcionalnost aplikacije, neophodno je da svi podsistemi budu aktivni, jer međusobna komunikacija i razmjena podataka zavise od njihove dostupnosti.

Inicijalizacija se sastoji iz više faza:

- **Priprema okruženja** – instalacija potrebnog softvera.
- **Instalacija zavisnosti** – učitavanje svih biblioteka potrebnih za rad aplikacije putem **Apache Maven** alata.
- **Konfiguracija sistema** – podešavanje konfiguracionih fajlova, uključujući povezivanje sa bazom podataka.
- **Inicijalizacija baze podataka** – primjena migracija i kreiranje administratorskog korisnika kroz izvršavanje odgovarajuće skripte.
- **Pokretanje razvojnih servera.**

Aplikacija u ovom režimu funkcionira lokalno, a svi podsistemi, uključujući backend API i frontend interfejs, moraju biti aktivni kako bi osigurali punu funkcionalnost i efikasnu razmjenu podataka.

2.8.2 Terminiranje

Sistem je dizajniran tako da se svaki njegov podsistem može ugasiti nezavisno od ostalih. Ovakav pristup pruža veću fleksibilnost prilikom održavanja, nadogradnje ili otklanjanja eventualnih problema.

Pri gašenju bilo kojeg dijela sistema treba voditi računa o pravilnom oslobađanju svih korištenih resursa:

- Konekcije prema bazi podataka - Spring Boot automatski zatvara aktivne konekcije prilikom zaustavljanja aplikacije
- Memorijski resursi - Sva zauzeta memorija biva oslobođena kako bi se izbjeglo curenje memorije.
- Mrežne konekcije - zatvaraju se otvoreni portovi i veze prema mreži.

Nakon gašenja, sistem je moguće ponovo pokrenuti bez bilo kakvih problema.

2.8.3 Otkazi

Sistem je projektovan tako da bude otporan na otkaze i da u slučaju grešaka ili neočekivanih kvarova reaguje na pouzdan i predvidiv način. U slučaju da se dogodi neki tehnički problem, sistem će prvenstveno izvršiti detekciju uzroka problema, a nakon toga izvršiti odgovarajuće mjere za njegovo otklanjanje.

Upotreba slojevite arhitekture omogućava da otkazi u okviru pojedinačnih podсистema ne utiču na funkcionisanje cijelog sistema. U slučaju da dođe do privremenog prekida u radu sa nekom od komponenata, korisniku će biti prikazana adekvatna poruka o trenutnoj nedostupnosti komponente, te obavještenje o planiranom vremenu oporavka.

Cilj je da se svi prekidi u radu svedu na minimum kako bismo obezbijedili pouzdan rad aplikacije i što bolje korisničko iskustvo.